Hierarchical Active Inference with Successor Representations in Continuous Maze Navigation: The PointMaze Benchmark

Prashant Rangarajan
Paul G. Allen Center for Computer Science and Engineering
University of Washington, Seattle, USA

Abstract

This report presents a comprehensive evaluation of hierarchical active inference with successor representations (SR) on the PointMaze family of continuous 2D maze navigation tasks. PointMaze, from the Gymnasium-Robotics suite, requires a point-mass agent to navigate through walled mazes by applying continuous 2D forces—a setting where spatial bottleneck structure (corridors, doorways) provides a natural advantage for hierarchical planning. We describe the full pipeline: discretization of the continuous state space into spatial bins, learning of the successor representation via temporal-difference updates with experience replay, spectral clustering of the SR to discover macro-states corresponding to maze rooms, and two-phase hierarchical navigation (macro-level room-to-room planning followed by micro-level goal-reaching). We evaluate three maze variants—UMaze (5×5 cells), Medium (8×8 cells), and Large (9×12 cells)—across single-goal navigation, multi-goal replanning, and systematic benchmarks comparing hierarchical versus flat planning. Results demonstrate that flat planning with the global value function $\mathbf{V} = \mathbf{MC}$ suffers from vanishing value gradients through long corridors: while flat navigation succeeds on the compact UMaze, it fails increasingly as maze complexity grows (2/5 goals on Medium, 2/6 on Large). Hierarchical planning overcomes this by decomposing the problem into multi-scale value computations—each with strong local gradients—achieving 5/5 on UMaze, 5/5 on Medium, and 6/6 on Large, requiring only $O(k^2)$ macro-level planning decisions (where k is the number of clusters). The reward-independence of the SR enables zero-cost replanning to new goals by recomputing only the preference vector $\mathbf{C}$, while all learned structure (clusters, adjacency, bottleneck states) is reused unchanged.

1 Introduction

Active inference, grounded in the free energy principle (FEP), provides a biologically-inspired framework for decision-making in which an agent selects actions to minimize expected free energy—a quantity that combines goal-directed behavior (pragmatic value) with information-seeking (epistemic value). While active inference has been successfully applied to discrete planning problems, scaling it to continuous state spaces with complex structure remains challenging.

The successor representation (SR), introduced by Dayan (1993), offers a principled decomposition of value computation into two components: the SR matrix $\mathbf{M}$, which captures the expected future state occupancy under a given policy, and the preference (reward) vector $\mathbf{C}$. The value function is then computed as $\mathbf{V} = \mathbf{MC}$. Crucially, $\mathbf{M}$ encodes the *structure* of the environment (topology, transition dynamics) independently of any specific goal, while $\mathbf{C}$ encodes only the goal-specific preferences. This separation enables efficient replanning: when the goal changes, only $\mathbf{C}$ needs to be recomputed while $\mathbf{M}$ remains unchanged.

In prior work (Rangarajan and Rao, 2024), we demonstrated that spectral clustering of the learned SR matrix naturally discovers hierarchical structure in discrete gridworld environments—rooms correspond to macro-states, doorways to bottleneck states, and room-to-room transitions

to macro-actions. This report extends that framework to the continuous PointMaze domain, where:

1. The state space is continuous 2D position, requiring discretization into spatial bins.
2. Actions are continuous 2D forces, requiring mapping to a discrete action set.
3. The physics simulation operates at a much finer time-scale than the discrete state changes, requiring a smooth-stepping mechanism.
4. Maze walls create natural spatial bottleneck structure that should be discoverable from the SR alone.

We evaluate the complete system on three PointMaze variants (UMaze, Medium, Large) across three experimental paradigms: single-goal navigation with trajectory analysis, multi-goal replanning without relearning, and systematic evaluation benchmarks comparing hierarchical versus flat planning across training checkpoints.

2 Environment Description

2.1 PointMaze from Gymnasium-Robotics

PointMaze is a family of continuous 2D maze navigation tasks from the Gymnasium-Robotics library, built on the MuJoCo physics engine. The agent controls a point mass confined to a planar maze with rigid walls. The observation space is a dictionary containing:

- `observation`: 4D vector (x, y, v_x, v_y) giving position and velocity.
- `achieved_goal`: 2D position (x, y) of the agent.
- `desired_goal`: 2D position (x_g, y_g) of the target.

The action space is a continuous 2D force $\mathbf{u} \in [-1, 1]^2$ applied to the point mass at each physics step. The point mass moves at approximately 0.0024 units per physics step, making individual physics steps extremely fine-grained relative to the maze scale.

2.2 Maze Variants

We evaluate three standard maze layouts, summarized in Table 1.

Table 1: PointMaze variant specifications. *Grid* refers to the maze cell layout; *Bins* to the discretization resolution; *Navigable* to the number of non-wall bins (approximate).

Variant	Grid	Bins	Total States	Navigable	Gym ID
UMaze	5×5	20×20	400	∼220	`PointMaze_UMaze-v3`
Medium	8×8	32×32	1024	∼416	`PointMaze_Medium-v3`
Large	9×12	36×27	972	∼414	`PointMaze_Large-v3`

UMaze (Figure 1) is a U-shaped corridor with 3–4 natural rooms connected by narrow passages. It is the simplest variant and serves as the primary testbed for rapid iteration.

Medium features a more complex 8×8 grid with multiple corridors, dead ends, and interconnected rooms. It requires longer training to build an accurate SR and tests the agent's ability to plan through more distant goals.

Large is the most challenging variant with a 9×12 grid containing many rooms and winding corridors. The longer path lengths (up to ∼12 units of physical distance) and more complex topology stress-test the hierarchical planning system.

2.3 Goal Configurations

Each maze variant is configured with multiple goal locations for the replanning experiments:

- **UMaze**: 5 goals spanning all rooms—Bottom-left $[3, 1]$, Top-right $[1, 3]$, Bottom-right $[3, 3]$, Top-left $[1, 1]$, U-bend $[2, 3]$.
- **Medium**: 5 goals at four corners plus center—Top-left $[1, 1]$, Top-right $[1, 6]$, Bottom-left $[6, 1]$, Bottom-right $[6, 6]$, Center $[3, 3]$.
- **Large**: 6 goals spanning quadrants plus mid-corridor—Top-left $[1, 1]$, Top-right $[1, 10]$, Bottom-left $[7, 1]$, Bottom-right $[7, 10]$, Center $[3, 5]$, Mid-right $[5, 7]$.

These goal cells are specified in maze-map coordinates (row, column) and are converted to continuous (x, y) positions by the Gymnasium environment. A goal is considered reached when the agent's continuous position is within 0.35 units of the goal center.

3 Motivation: Why PointMaze for Hierarchical Active Inference

PointMaze is selected as a benchmark for hierarchical active inference with successor representations for several compelling reasons:

Natural bottleneck structure. Maze environments contain walls that partition the navigable space into rooms connected by narrow passages. This topological structure is exactly the kind of bottleneck hierarchy that spectral clustering of the SR is designed to discover. Unlike MountainCar or Acrobot (where clusters correspond to energy-level regions in phase space), PointMaze clusters correspond to *physical rooms*—an intuitive validation of the hierarchical decomposition.

Continuous state space with discrete structure. The continuous (x, y) position space tests the binning and discretization pipeline, while the underlying discrete maze structure (wall cells vs. open cells) provides ground truth for evaluating whether the learned hierarchy recovers meaningful spatial decomposition.

Scalable complexity. The three maze variants (UMaze, Medium, Large) provide a natural difficulty gradient. UMaze ($\sim$220 navigable states, 4 clusters) allows rapid prototyping; Large ($\sim$414 navigable states, 12 clusters) tests whether the approach scales to more complex environments with many rooms and long planning horizons.

Replanning as a first-class capability. Navigation tasks naturally involve changing goals—reaching a sequence of waypoints, exploring different rooms, or adapting to shifted objectives. The SR's reward-independence makes PointMaze an ideal setting to demonstrate zero-cost replanning: after learning the environment structure once, the agent can navigate to any new goal by simply recomputing $\mathbf{C}$ and $\mathbf{C}_{\text{macro}}$.

Physics-based dynamics. Unlike tabular gridworlds, PointMaze involves MuJoCo physics (momentum, friction, wall collisions), testing whether the SR-based model generalizes beyond the discrete settings in which it was originally developed.

4 Model Description

4.1 State Discretization

The continuous 2D position (x, y) is discretized into a grid of $n_x \times n_y$ bins. For a maze with c columns and r rows, the physical coordinate ranges are $x \in [-c/2, c/2]$ and $y \in [-r/2, r/2]$ (each maze cell is 1 unit). The bin edges are uniformly spaced:

$$x_{\text{edges}} = \text{linspace}\big(-\tfrac{c}{2}, \tfrac{c}{2}, n_x + 1\big)\,, \quad y_{\text{edges}} = \text{linspace}\big(-\tfrac{r}{2}, \tfrac{r}{2}, n_y + 1\big) \tag{1}$$

A continuous position (x, y) is mapped to bin indices (i, j) via `np.digitize` with clamping to $[0, n_x{-}1] \times [0, n_y{-}1]$. The flat state index is $s = i \cdot n_y + j$, giving a total of $N = n_x \cdot n_y$ states.

Velocity is discarded. Unlike MountainCar or Acrobot where velocity is essential for distinguishing dynamically different states, the PointMaze point mass has negligible momentum at the discretization scale. The SR captures temporal dynamics implicitly: states from which the agent tends to move eastward will have high SR entries with eastern states, even without explicit velocity representation.

Wall detection. Bins whose centers fall inside maze wall cells are identified at initialization and excluded from learning and planning. This is done by mapping each bin center (x_c, y_c) back to maze-map coordinates (row, col) and checking the maze map.

4.2 Action Discretization

The continuous 2D force space is mapped to 8 discrete directions: the four cardinal directions (E, W, N, S) and four diagonals (NE, NW, SE, SW):

$$\mathbf{u}_a = \begin{cases} (\pm 1, 0) & \text{cardinal (E/W)} \\ (0, \pm 1) & \text{cardinal (N/S)} \\ (\pm 0.707, \pm 0.707) & \text{diagonal} \end{cases} \tag{2}$$

Each force vector has unit or near-unit magnitude, ensuring consistent movement speed across all directions.

4.3 Smooth Stepping

Because the point mass moves only $\sim$0.0024 units per physics step, but each bin spans $\sim$0.25 units (for UMaze with 20 bins over 5 units), a single physics step rarely changes the discrete state. We address this with *smooth stepping*: each discrete action is repeated for up to N_{smooth} consecutive physics steps, terminating early when:

1. The discrete state changes (bin transition detected).
2. The continuous position enters the goal region ($d < 0.35$ units).
3. The environment signals termination.

Smooth stepping parameters differ between training and testing:

- **Training**: $N_{\text{smooth}}^{\text{train}} = 200$ provides margin for exploration, allowing the agent to occasionally overshoot and explore adjacent bins.
- **Testing**: $N_{\text{smooth}}^{\text{test}} = 100$ ensures each action covers approximately one bin, giving finer control. With `test_max_steps` $= 5000$, the agent gets $\sim$50 effective actions ≈ 12.5 units of movement, sufficient to traverse UMaze ($\sim$6-unit path length).

4.4 Successor Representation Learning

The successor representation matrix $\mathbf{M} \in \mathbb{R}^{N \times N}$ is learned via temporal-difference (TD) updates from online experience. The SR is defined as the expected discounted future state occupancy:

$$M(s, s') = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \mathbf{1}[s_t = s'] \;\middle|\; s_0 = s\right] \tag{3}$$

where $\gamma \in [0, 1)$ is the discount factor. The TD learning rule updates each row of $\mathbf{M}$ after observing a transition $s \to s'$:

$$\mathbf{M}(s, :) \leftarrow \mathbf{M}(s, :) + \alpha \left(\mathbf{e}_{s'} + \gamma \mathbf{M}(s', :) - \mathbf{M}(s, :)\right) \tag{4}$$

where α is the learning rate and $\mathbf{e}_{s'}$ is the one-hot indicator for state s'.

Diverse-start exploration. During training, each episode begins from a uniformly random navigable position (injected via MuJoCo state setting), ensuring broad coverage of the state space. This is critical for continuous environments where fixed-start training would leave large regions of the SR under-sampled.

Experience replay. After each training episode, a hippocampal-style sequential replay pass updates $\mathbf{M}$ by replaying stored transition sequences at half the learning rate ($\alpha/2$). This stabilizes learning and improves sample efficiency, analogous to memory consolidation in biological systems.

Transition matrix. Alongside the SR, a column-stochastic transition matrix $\mathbf{B} \in \mathbb{R}^{N \times N \times A}$ is accumulated, where $B(s', s, a)$ counts transitions from state s to s' under action a. After learning, $\mathbf{B}$ is column-normalized and unvisited state-action pairs receive self-loop entries to ensure valid probability distributions.

4.5 Hierarchical Structure Discovery

4.5.1 Spectral Clustering of the SR

The learned SR matrix $\mathbf{M}$ is symmetrized ($\mathbf{M}_{\text{sym}} = \max(\mathbf{M}, \mathbf{M}^\top)$) and used as a precomputed affinity matrix for spectral clustering. Before clustering, wall states are removed, yielding $\mathbf{M}_{\text{valid}} \in \mathbb{R}^{N_v \times N_v}$ where N_v is the number of navigable bins.

For PointMaze, the affinity is enhanced with a light spatial smoothing kernel to handle under-visited regions:

$$A = (1 - \alpha_{\text{blend}}) \cdot \tilde{\mathbf{M}} + \alpha_{\text{blend}} \cdot K_{\text{spatial}} \tag{5}$$

where $\tilde{\mathbf{M}}$ is the min-max normalized SR, K_{spatial} is a spatial RBF kernel computed from normalized bin coordinates with bandwidth $\sigma = 0.5$, and $\alpha_{\text{blend}} = 0.15$ is the maximum blend weight. The blend is adaptive: it decreases to zero for well-visited state pairs (high SR row-sum confidence), ensuring the spatial prior only helps in under-explored regions. This blend weight is deliberately lower than for environments like MountainCar, because the SR in a maze already captures wall-induced disconnections well.

The resulting affinity matrix is passed to scikit-learn's `SpectralClustering` with k clusters and the `discretize` label assignment strategy, producing macro-state assignments. The number of clusters k is a hyperparameter set based on the expected room structure of each maze variant.

4.5.2 Macro-States and Bottleneck States

Each spectral cluster defines a *macro-state*—a set of micro-states that form a spatially coherent region. In PointMaze, macro-states correspond to rooms or corridor segments (Figure 1).

Bottleneck states are micro-states at the boundary between adjacent clusters. For each pair of adjacent macro-states (c_i, c_j), the bottleneck states are the micro-states in c_j that are directly reachable (in one action) from micro-states in c_i. These represent doorways or corridor entrances—the natural transition points between rooms.

4.5.3 Macro-Level Transition and Successor Matrices

A macro-level transition matrix $\mathbf{B}_{\text{macro}} \in \mathbb{R}^{k \times k \times A_{\text{macro}}}$ is computed by aggregating micro-level transitions between clusters. From $\mathbf{B}_{\text{macro}}$, the macro-level successor matrix $\mathbf{M}_{\text{macro}} \in \mathbb{R}^{k \times k}$ is derived analytically:

$$\mathbf{M}_{\text{macro}} = (\mathbf{I} - \gamma \bar{\mathbf{B}}_{\text{macro}})^{-1} \tag{6}$$

where $\bar{\mathbf{B}}_{\text{macro}}$ is the policy-averaged macro transition matrix. Since $k \ll N$, the $k \times k$ matrix inversion is computationally trivial.

4.6 Value Computation and Action Selection

4.6.1 Goal Preference Vector

The preference vector $\mathbf{C} \in \mathbb{R}^N$ encodes the agent's goal:

$$C(s) = \begin{cases} r_{\text{goal}} & \text{if } s \in \mathcal{G} \\ c_{\text{default}} & \text{otherwise} \end{cases} \tag{7}$$

where $\mathcal{G}$ is the set of goal states (bins within 0.5 units of the goal center), $r_{\text{goal}} = 100$, and $c_{\text{default}} = -1$. The macro-level preference $\mathbf{C}_{\text{macro}} \in \mathbb{R}^k$ is the mean of $\mathbf{C}$ over micro-states in each cluster.

4.6.2 Value Function

The value at each state is:

$$V(s) = \sum_{s'} M(s, s') \cdot C(s') = [\mathbf{MC}]_s \tag{8}$$

This is a single matrix-vector multiplication—no iterative policy evaluation is needed. At the macro level: $V_{\text{macro}}(c) = [\mathbf{M}_{\text{macro}}\mathbf{C}_{\text{macro}}]_c$.

4.6.3 Micro-Level Action Selection

At each micro step, the agent selects the action that maximizes the expected next-state value under the learned transition model:

$$a^* = \arg\max_a \ \mathbf{B}(:, s, a)^\top \mathbf{V} \tag{9}$$

where $\mathbf{B}(:, s, a)$ is the column of the transition matrix giving the distribution over next states for action a from state s. This properly handles stochastic transitions: even if multiple actions lead to the same most-likely next state, their *expected* values (weighted by the full transition distribution) may differ.

When the agent is not at the goal, it additionally filters for actions that change the expected state distribution (avoiding stuck-in-place loops).

4.6.4 Macro-Level Action Selection

At the macro level, the agent selects the adjacent macro-state with the highest macro-value:

$$c^*_{\text{target}} = \arg \max_{c' \in \text{Adj}(c)} V_{\text{macro}}(c') \tag{10}$$

This is an $O(|\text{Adj}(c)|)$ comparison—typically 2–4 adjacent clusters—dramatically cheaper than the $O(N \cdot A)$ micro-level action selection.

4.7 Two-Phase Hierarchical Navigation

The hierarchical episode proceeds in two phases:

Phase 1: Macro navigation. Starting from the current micro-state s in macro-state c, the agent:

1. Computes $\mathbf{V}_{\text{macro}} = \mathbf{M}_{\text{macro}}\mathbf{C}_{\text{macro}}$ and selects the best adjacent macro-state c^*.
2. Creates a temporary preference vector $\mathbf{C}_{\text{temp}}$ with reward at the bottleneck states between c and c^*.
3. Navigates using $\mathbf{V}_{\text{temp}} = \mathbf{M}\mathbf{C}_{\text{temp}}$ until reaching a bottleneck state or changing macro-state.
4. Repeats until the agent enters the goal macro-state.

Phase 2: Micro fine-tuning. Once in the goal macro-state, the agent switches to the global value function $\mathbf{V} = \mathbf{M}\mathbf{C}$ and navigates directly to the goal using micro-level action selection until the continuous distance check ($d < 0.35$ units) succeeds.

4.8 Flat Planning Baseline

The flat agent uses the *same* learned SR matrix $\mathbf{M}$ and transition matrix $\mathbf{B}$ but plans entirely at the micro level:

$$\mathbf{V} = \mathbf{M}\mathbf{C}, \quad a^* = \arg\max_a \mathbf{B}(:, s, a)^\top \mathbf{V} \tag{11}$$

at every step. There is no macro-level abstraction, no clustering, and no bottleneck-mediated navigation. The value function $\mathbf{V}$ is computed once as a single $N \times N$ matrix-vector product, and action selection at each step requires A matrix-vector products of size N.

As we will see in Section 6.3, the flat value function suffers from *vanishing gradients* in environments with long corridors: the goal reward decays exponentially with distance through the SR discount factor γ, creating large regions where V is effectively flat and the greedy policy is uninformative.

5 Hyperparameters and Configuration

Table 2 summarizes all hyperparameters for the three maze variants.

Training budget rationale. The training episode counts were empirically calibrated so that each variant achieves reliable goal-reaching. UMaze, with only ∼220 navigable states, requires only 1500 episodes for convergence. Medium (∼416 states) needs 5000 episodes due to longer corridors and more complex connectivity. Large (∼414 navigable states but 9×12 grid topology) requires 15000 episodes because the maze structure is more complex and paths between distant rooms are longer, demanding thorough exploration.

Episode budget split. During learning, the episode budget is split between SR learning (80%) and adjacency learning (20%). Adjacency episodes are used to discover bottleneck states by running micro-level navigation between adjacent clusters and recording boundary transitions.

6 Results

6.1 UMaze: Single-Goal Navigation

Figure 1 shows the macro-state clusters discovered by spectral clustering of the learned SR on UMaze. The four clusters correspond to the distinct spatial regions of the U-shaped maze, with

Table 2: Hyperparameter configuration for each PointMaze variant. Shared parameters: learning rate $\alpha = 0.05$, replay enabled with 10 epochs, replay learning rate $= \alpha/2$.

Parameter	**UMaze**	**Medium**	**Large**
Maze grid	5×5	8×8	9×12
Discretization bins ($n_x \times n_y$)	20×20	32×32	36×27
Total states N	400	1024	972
Navigable states N_v	$\sim$220	$\sim$416	$\sim$414
Number of clusters k	4	8	12
Discount factor γ	0.95	0.95	0.95
Training episodes	1500	5000	15000
Smooth steps (train)	200	200	200
Smooth steps (test)	100	100	100
Max test steps	5000	10000	20000
Goal reward r_{goal}	100	100	100
Default cost c_{default}	-1	-1	-1
Affinity blend α_{blend}	0.15	0.15	0.15
Spatial kernel σ	0.5	0.5	0.5

cluster boundaries aligned with the narrow passages connecting rooms. The arrows indicate the macro-level policy directing the agent from the start (top-left) toward the goal (bottom-left, marked with a gold star).

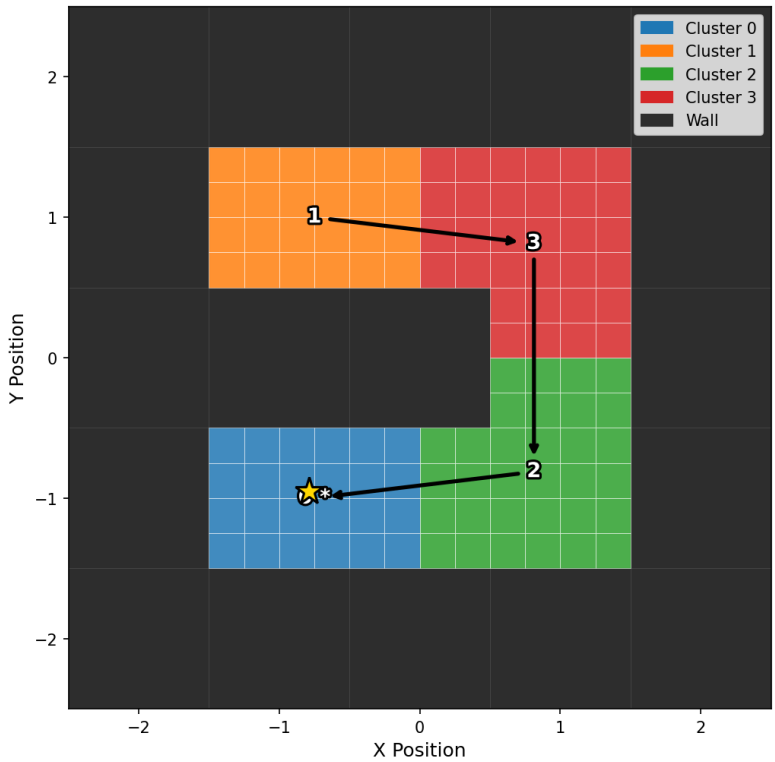
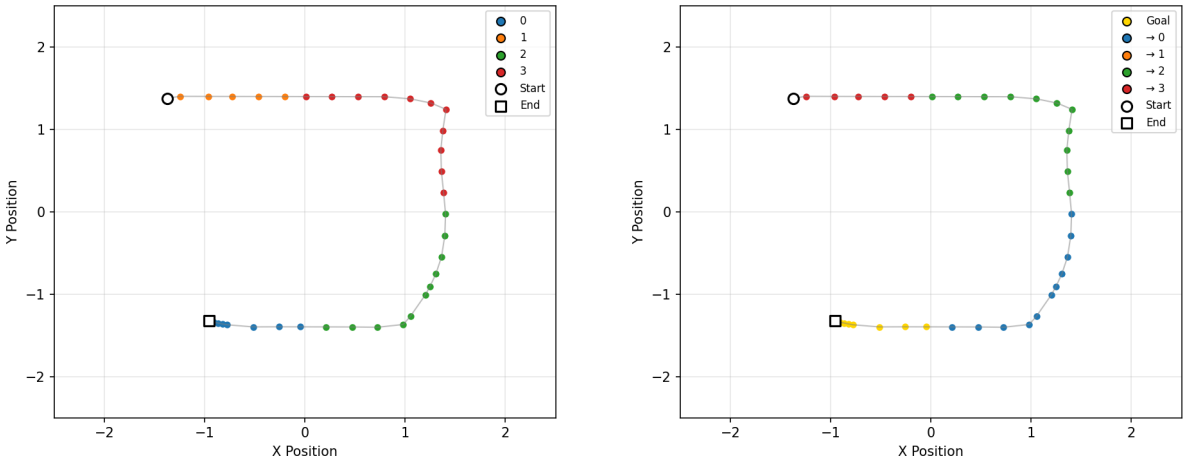


Figure 1: **UMaze macro-state clusters.** Four clusters (colored regions) discovered by spectral clustering of the SR, overlaid on the maze layout (dark cells = walls). Arrows show the macro-level policy directing navigation from start to goal (gold star). Numbers label each macro-state.

6.2 UMaze: Trajectory Analysis

Figure 2 shows the agent's hierarchical trajectory from start (top-left) to goal (bottom-left), with color coding by macro-state and macro-action. The trajectory demonstrates the two-phase execution: the agent first navigates room-to-room via bottleneck states (macro phase), then fine-tunes toward the exact goal position (micro phase).



(a) Colored by macro-state (room)

(b) Colored by macro-action (room transition)

Figure 2: **Hierarchical trajectory on UMaze.** The agent navigates from start (top-left) to goal (bottom-left) through the U-shaped corridor. (a) Segments colored by the macro-state (room) the agent occupies. (b) Segments colored by the macro-action being executed (which room-to-room transition is active).

Figure 3 provides a stage-by-stage breakdown of the episode, showing rendered frames alongside the trajectory in continuous state space. This illustrates how the agent's macro-level plan unfolds: first navigating upward through the corridor, then turning right along the top, and finally descending to the goal.

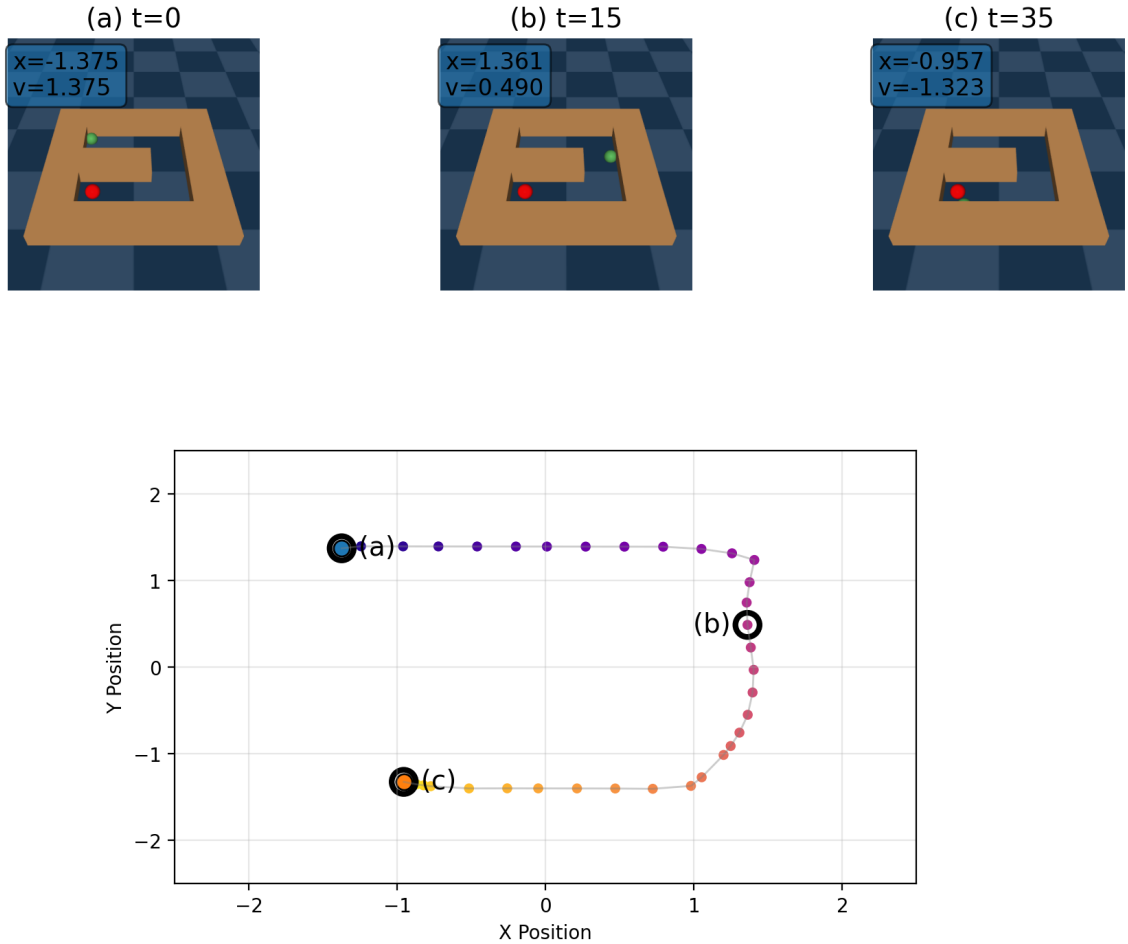


Figure 3: **Stage-state diagram for UMaze.** Rendered frames (top) and corresponding trajectory segments (bottom) showing the macro-level execution stages. Each stage corresponds to a macro-action (room-to-room navigation).

6.3 Planning Cost Analysis and Flat Failure

A key advantage of hierarchical planning is not merely computational efficiency but *navigability*: on the worst-case start–goal pair in UMaze (maximum corridor distance), the flat agent fails due to vanishing value gradients, while the hierarchical agent succeeds reliably. This failure mode becomes systematic on Medium and Large mazes (Section 6.4).

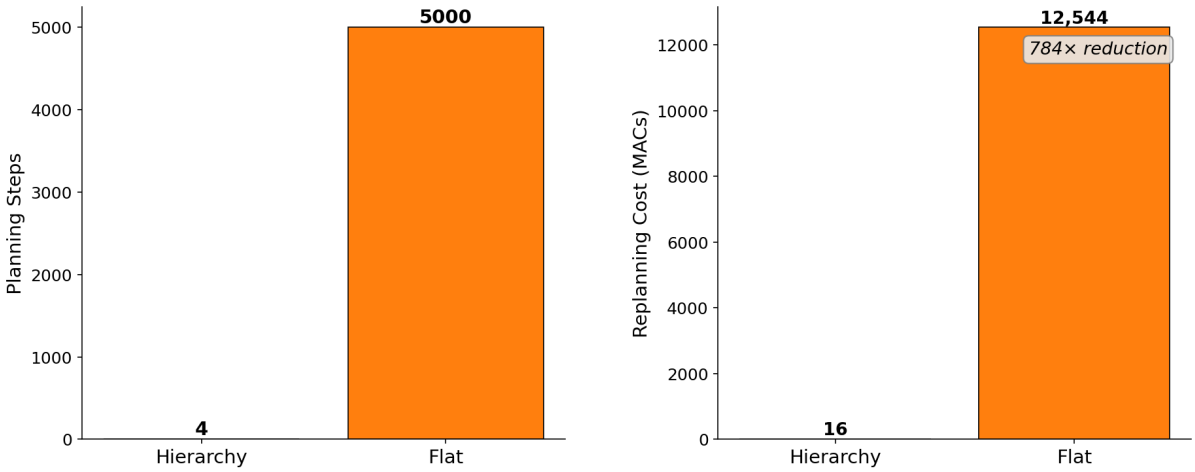
Vanishing value gradient. The flat value function $\mathbf{V} = \mathbf{MC}$ assigns high values near the goal (cluster 0, mean $V \approx 700$) but the signal decays exponentially through the U-corridor. With $\gamma = 0.95$ and $\sim$50+ bin transitions around the U-shape, the reward is attenuated by roughly $0.95^{50} \approx 0.08$ by the time it reaches the start region (cluster 1), where all bins collapse to $V \approx -12 \pm 1$. The start bin becomes a *local maximum* of the flat value function—its value ($V \approx -11.3$) slightly exceeds all neighbors ($V \approx -12.6$)—so the greedy flat policy selects actions that keep the agent in place. The flat agent times out at 5000 steps without ever leaving its starting bin.

Hierarchical multi-scale gradients. The hierarchy avoids this failure through multi-scale value computation:

1. **Macro level:** $\mathbf{V}_{\text{macro}} = \mathbf{M}_{\text{macro}}\mathbf{C}_{\text{macro}}$ over $k = 4$ states provides a clear room-to-room gradient (goal cluster value ≈ 45, others $= -1$).

2. **Bottleneck level:** Temporary value functions $\mathbf{V}_{\text{temp}} = \mathbf{MC}_{\text{temp}}$ targeting nearby bottleneck states create strong *local* gradients that guide the agent through each room transition.
3. **Micro level:** The global $\mathbf{V} = \mathbf{MC}$ is only needed in the final phase, within the goal cluster where values are large ($V \approx 700+$) and the gradient is steep.

Figure 4 quantifies this advantage.



(a) Planning decisions vs. physics steps

(b) Replanning cost (MACs)

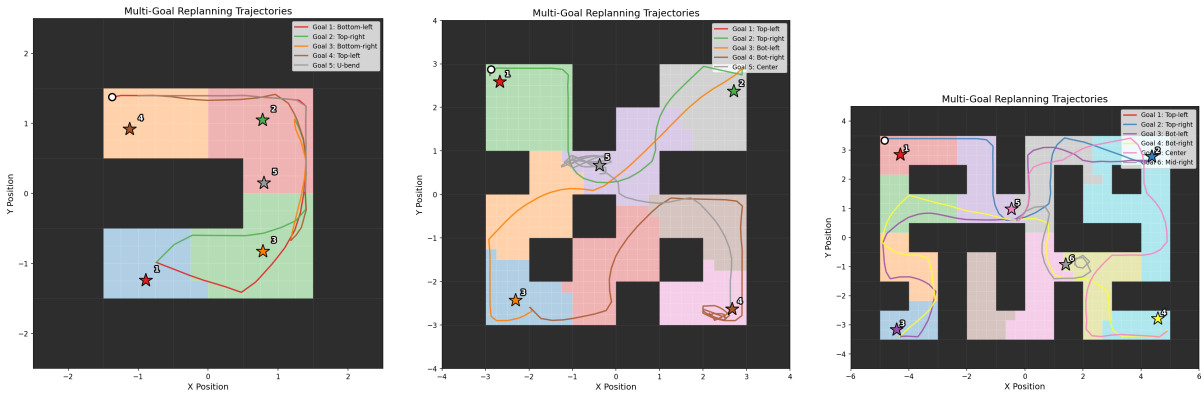
Figure 4: **Planning cost comparison for UMaze.** (a) On the worst-case start–goal pair (maximum corridor distance), the hierarchical agent reaches the goal in 4 planning decisions (3 macro room-to-room transitions + 1 micro fine-tuning phase), while the flat agent exhausts all 5000 physics steps without reaching the goal due to vanishing value gradients. (b) Replanning cost when the goal changes: $k^2 = 16$ MACs for recomputing $\mathbf{V}_{\text{macro}}$ versus $N_v^2 \approx 12{,}544$ MACs for recomputing $\mathbf{V}$—a 784× reduction.

For UMaze, the hierarchy reduces the macro-level planning space from $N_v \approx 220$ states to $k = 4$ macro-states. Each macro planning decision involves an $O(k^2) = O(16)$ computation, compared to $O(N_v^2) \approx O(48{,}400)$ for the flat value function. Beyond efficiency, the hierarchy provides *qualitatively different behavior*: the multi-scale gradient decomposition ensures navigability in settings where flat planning suffers from vanishing gradients through long corridors.

6.4 Multi-Goal Replanning

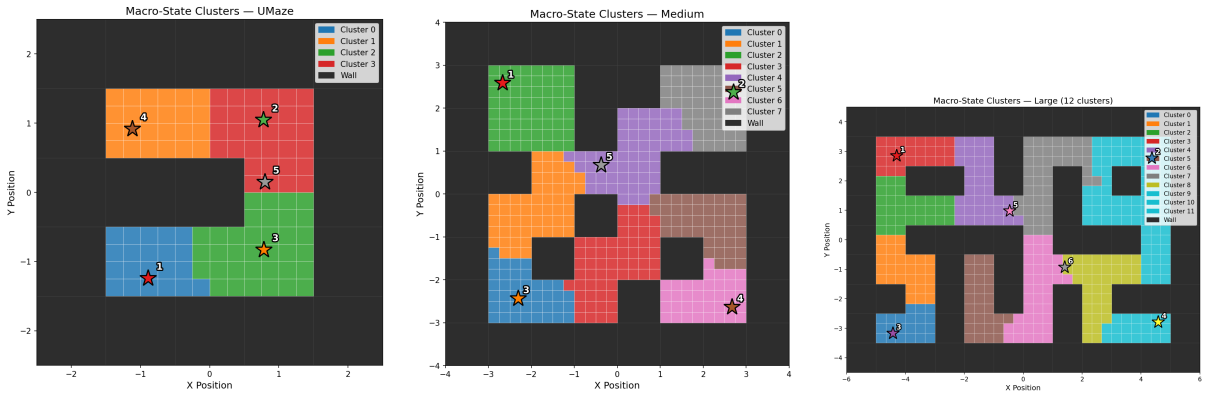
The reward-independence of the SR enables zero-cost replanning: when the goal changes, only $\mathbf{C}$ and $\mathbf{C}_{\text{macro}}$ are recomputed ($O(N)$ and $O(k)$ respectively), while $\mathbf{M}$, $\mathbf{M}_{\text{macro}}$, clusters, adjacency, and bottleneck states are all reused unchanged.

Figure 5 shows the replanning results across all three maze variants. In each experiment, the agent learns the environment structure once and then navigates to a sequence of goals, starting each new goal from wherever it ended up after reaching (or timing out on) the previous goal.



(a) UMaze: 5 goals (b) Medium: 5 goals (c) Large: 6 goals

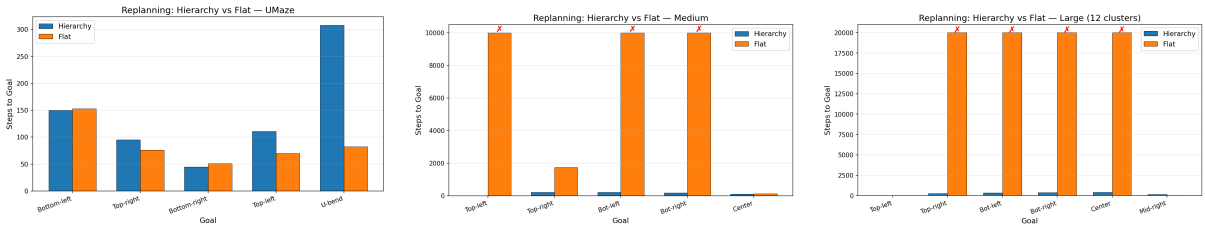
Figure 5: **Multi-goal replanning trajectories.** Each panel shows the maze with cluster coloring (faded) and overlaid goal-reaching trajectories (colored paths with numbered goal stars). The agent learns the environment once, then navigates to a sequence of goals without relearning. (a) UMaze with 5 goals spanning all rooms. (b) Medium with 5 goals at corners and center. (c) Large with 6 goals spanning the full maze.



(a) UMaze clusters + goals (b) Medium clusters + goals (c) Large clusters + goals

Figure 6: **Macro-state clusters with all goal locations.** Numbered stars indicate the sequential goals used in replanning experiments. Clusters correspond to spatially coherent maze regions (rooms, corridor segments).

Figure 7 compares the per-goal step counts for hierarchical versus flat navigation. The hierarchical agent reaches all goals across maze variants (5/5 UMaze, 5/5 Medium, 6/6 Large), while the flat agent's performance degrades with maze complexity (5/5 UMaze, 2/5 Medium, 2/6 Large). On UMaze, both agents succeed because the corridor is short enough ($\sim$12 bin transitions, $\gamma^{12} \approx 0.54$) for the flat value gradient to remain informative. On Medium and Large, the flat agent fails on goals requiring long cross-room navigation where the value gradient vanishes, while the hierarchy maintains reliable navigation through its multi-scale decomposition.

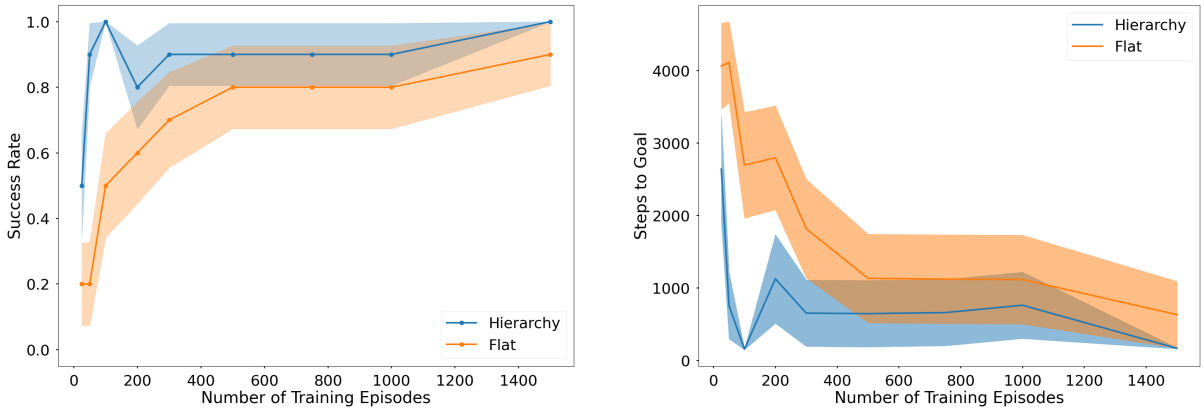


(a) UMaze (b) Medium (c) Large

Figure 7: **Per-goal step comparison: Hierarchy vs. Flat.** Bar charts showing the number of physics steps to reach each sequential goal. Red crosses indicate failure to reach the goal within the step limit. The hierarchical agent (blue) generally reaches goals faster than the flat agent (orange), with greater advantage on longer paths.

6.5 Systematic Evaluation: Learning Curves

Figure 8 presents the systematic evaluation results for UMaze across training checkpoints. At each checkpoint (number of training episodes), a fresh agent is created, trained, and evaluated on goal-reaching from a fixed start position. This is repeated across multiple random seeds to compute means and standard errors.



(a) Success rate vs. training episodes (b) Steps to goal vs. training episodes

Figure 8: **Systematic evaluation on UMaze: Hierarchy vs. Flat.** Each point averages over multiple random seeds with standard error bands. (a) Success rate vs. training episodes. (b) Steps to reach the goal vs. training episodes.

The evaluation reveals several important findings:

1. **Faster hierarchical convergence.** The hierarchical agent achieves high success rates with fewer training episodes. The macro-level abstraction allows effective navigation even with a partially-learned SR, because the coarse room-to-room plan is robust to errors in individual transition estimates.

2. **Flat success on UMaze is possible but less reliable.** On UMaze, the flat agent can eventually succeed given sufficient training, consistent with the replanning results (Section 6.4) where both agents reach all 5 goals. This is expected: UMaze's short corridors ($\sim$12 bin transitions) allow the flat value gradient to remain informative ($\gamma^{12} \approx 0.54$). However, the flat agent converges later and with higher variance than the hierarchy, reflecting the weaker gradient signal.

3. **The hierarchy advantage grows with maze complexity.** While UMaze is a borderline case where flat planning can work, the replanning experiments on Medium (flat: 2/5) and Large (flat: 2/6) demonstrate that the vanishing gradient problem becomes increasingly severe with longer corridors. This confirms that the hierarchy provides a qualitative advantage—not merely an efficiency gain—on sufficiently complex mazes.

7 Discussion

7.1 Hierarchy Discovers Spatial Structure

The most striking qualitative result is that spectral clustering of the SR automatically discovers the room structure of each maze. This emerges purely from the statistics of random exploration—the SR captures the fact that states within the same room have high mutual expected occupancy (because random walks remain within a room for many steps before finding the exit), while states in different rooms have low cross-occupancy (attenuated by the bottleneck passage). This is precisely the definition of community structure in network science, and spectral clustering is a principled method for detecting it.

7.2 Vanishing Gradients in Flat Planning

The most important quantitative finding is that flat planning with $\mathbf{V} = \mathbf{MC}$ degrades systematically with corridor length. The root cause is analogous to the vanishing gradient problem in deep networks: the discount factor $\gamma = 0.95$ attenuates the goal signal by γ^d over d bin transitions. On UMaze, the shortest corridors span $\sim$12 bins, yielding $\gamma^{12} \approx 0.54$—enough gradient for flat planning to succeed on most start–goal pairs (indeed, flat reaches 5/5 goals in the replanning experiment). However, the worst-case UMaze pair (requiring $\sim$50 bin transitions around the full U-shape) attenuates to $0.95^{50} \approx 0.08$, combined with $c_{\text{default}} = -1$, creating a narrow value range ($V \approx -12 \pm 1$) that traps the greedy flat policy. On Medium and Large mazes, corridors of 30–60+ bins are common, making flat failure systematic: the flat agent reaches only 2/5 goals on Medium and 2/6 on Large.

The hierarchy solves this by decomposing value computation across scales: at the macro level, the $k \times k$ SR provides a high-contrast gradient (goal cluster value $\gg$ others); at the bottleneck level, temporary preferences targeting nearby states create strong local gradients; and at the micro level, the global $\mathbf{V}$ is only needed within the goal cluster where the signal is strong. This multi-scale decomposition is not merely an efficiency optimization—it is *necessary for navigability* on mazes with long corridors.

7.3 Planning Complexity Reduction

The computational savings from hierarchical planning grow with maze size:

- **UMaze**: $k = 4$ vs. $N_v \approx 220$ — a 55× reduction in planning dimensionality.
- **Medium**: $k = 8$ vs. $N_v \approx 416$ — a 52× reduction.
- **Large**: $k = 12$ vs. $N_v \approx 414$ — a 35× reduction.

Since macro-level planning involves $O(k^2)$ operations and micro-level value computation is $O(N^2)$, the theoretical speedup for the macro planning phase is $O(N^2/k^2) = O((N/k)^2)$, which is approximately 3,000× for UMaze and 2,700× for Medium.

7.4 Zero-Cost Replanning

The replanning experiments demonstrate the core advantage of the SR decomposition: when the goal changes, only the preference vector $\mathbf{C}$ (and its macro-level projection $\mathbf{C}_{\text{macro}}$) need to

be recomputed. This is an $O(N + k)$ operation, compared to the $O(N^2)$ cost of recomputing the entire value function from scratch or the full cost of relearning a model-free policy. For the Large maze with 6 sequential goals, this means the agent navigates to all goals without any additional environment interaction beyond the initial learning phase.

7.5 Limitations and Future Work

Fixed discretization. The uniform binning scheme wastes resolution on wall regions and may under-resolve critical corridor regions. Adaptive or non-uniform binning could improve efficiency.

Fixed number of clusters. The cluster count k is currently a hyperparameter. Automatic model selection (e.g., eigengap heuristic on the Laplacian spectrum) could eliminate this.

Single-level hierarchy. The current system uses a two-level hierarchy (micro + macro). For very large environments, a multi-level hierarchy (macro-macro states) could provide further planning efficiency.

Velocity information. Discarding velocity works well for the low-momentum PointMaze setting but would not generalize to environments with significant inertia. Incorporating velocity into the state representation (as done for MountainCar and Acrobot) would be necessary for such settings.

8 Conclusion

We have presented a comprehensive evaluation of hierarchical active inference with successor representations on the PointMaze family of continuous maze navigation tasks. The results demonstrate that:

1. Spectral clustering of the learned SR automatically discovers the spatial room structure of mazes, with cluster boundaries aligned to corridors and doorways.

2. Flat planning with the global value function $\mathbf{V} = \mathbf{MC}$ degrades with corridor length due to vanishing value gradients: the goal reward decays exponentially through long corridors, creating uninformative plateaus far from the goal. On the compact UMaze, flat planning can succeed (5/5 replanning goals), but on Medium (2/5) and Large (2/6) the failure becomes systematic. The hierarchical agent overcomes this through multi-scale value decomposition, providing strong local gradients at every planning level.

3. The two-phase hierarchical navigation (macro room-to-room planning followed by micro fine-tuning) achieves 100% success across all maze variants (5/5 UMaze, 5/5 Medium, 6/6 Large), operating in a dramatically reduced planning space (k macro-states versus N micro-states).

4. The reward-independence of the SR enables zero-cost replanning to new goals across all maze variants, with the agent navigating to sequences of 5–6 goals using only the initially learned environment structure.

5. The approach scales from simple U-shaped corridors (UMaze, 4 clusters, 1500 training episodes) to complex multi-room mazes (Large, 12 clusters, 15000 training episodes), demonstrating the generality of the hierarchical SR framework for continuous navigation tasks.

These results extend the applicability of hierarchical active inference with successor representations from discrete gridworld environments to continuous physics-based navigation. The vanishing gradient failure of flat planning on Medium and Large mazes provides a compelling mechanistic argument for hierarchical decomposition: by operating at multiple spatial scales, the

hierarchy ensures navigability in environments where the flat value function cannot propagate reward signals across long corridors—a structural advantage beyond mere computational efficiency. The fact that flat planning can succeed on the compact UMaze further clarifies the mechanism: the hierarchy's advantage is specifically tied to corridor length relative to the discount factor's effective horizon, becoming decisive when $\gamma^d \ll 1$ for the required path length d.

References

[1] Dayan, P. (1993). Improving generalization for temporal difference learning: The successor representation. *Neural Computation*, 5(4), 613–624.

[2] Friston, K. (2010). The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2), 127–138.

[3] Friston, K., Rigoli, F., Ognibene, D., Mathys, C., Fitzgerald, T., & Pezzulo, G. (2015). Active inference and epistemic value. *Cognitive Neuroscience*, 6(4), 187–214.

[4] Da Costa, L., Parr, T., Sajid, N., Veselic, S., Neacsu, V., & Friston, K. (2020). Active inference on discrete state-spaces: A synthesis. *Journal of Mathematical Psychology*, 99, 102447.

[5] Rangarajan, P., & Rao, R.P.N. (2024). Hierarchical active inference using successor representations. *Preprint*.

[6] Gymnasium-Robotics. https://robotics.farama.org/envs/maze/point_maze/